Tools Setup

In [ ]:
```
import pathlib, os, sys, operator, re, datetime
from functools import reduce
import numpy as np
import tensorflow as tf
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow import keras
from keras import Model
import tensorflow_datasets as tfds
from tiny_imagenet import TinyImagenetDataset

# Enable or disable GPU
# To fully disable it, we need to hide all GPU devices from Tensorflow
# Make sure GPU is disabled for this inference part of the lab
ENABLE_GPU = True
# tf.debugging.set_log_device_placement(True)

if not ENABLE_GPU:
    tf.config.set_visible_devices([], 'GPU')

# Print Python and TF version, and where we are running
print(f'Running on Python Version: {sys.version}')
print(f'Using Tensorflow Version: {tf. __version__}')
if not tf.config.experimental.list_physical_devices("GPU"):
    print('Running on CPU')
else:
    print(f'Using GPU at: {tf.test.gpu_device_name()} (of {len(tf.config.
```

Dataset Inspection

In [ ]:
```
# This cell imports our dataset.

# Original Source: https://github.com/ksachdeva/tiny-imagenet-tfds
# Class Version Source: https://github.com/duweisu/tiny-imagenet-tfds
# Setup our dataset
# ---------------------------------------------------------

tiny_imagenet_builder = TinyImagenetDataset()

# this call (download_and_prepare) will trigger the download of the datas
# and preparation (conversion to tfrecords)
#
# This will be done only once and on next usage tfds will
# use the cached version on your host.
tiny_imagenet_builder.download_and_prepare(download_dir="~/tensorflow-dat

# class_names = tiny_imagenet_builder.info.features['label'].names
ds = tiny_imagenet_builder.as_dataset()
ds_train, ds_val = ds["train"], ds["validation"]
assert(isinstance(ds_train, tf.data.Dataset))
assert(isinstance(ds_val, tf.data.Dataset))
```

```
# Training Dataset
ds_train = ds_train.shuffle(1024).prefetch(tf.data.AUTOTUNE)

# Validation Dataset
ds_val = ds_val.shuffle(1024).prefetch(tf.data.AUTOTUNE)

# Dataset metadata
ds_info = tiny_imagenet_builder.info
```

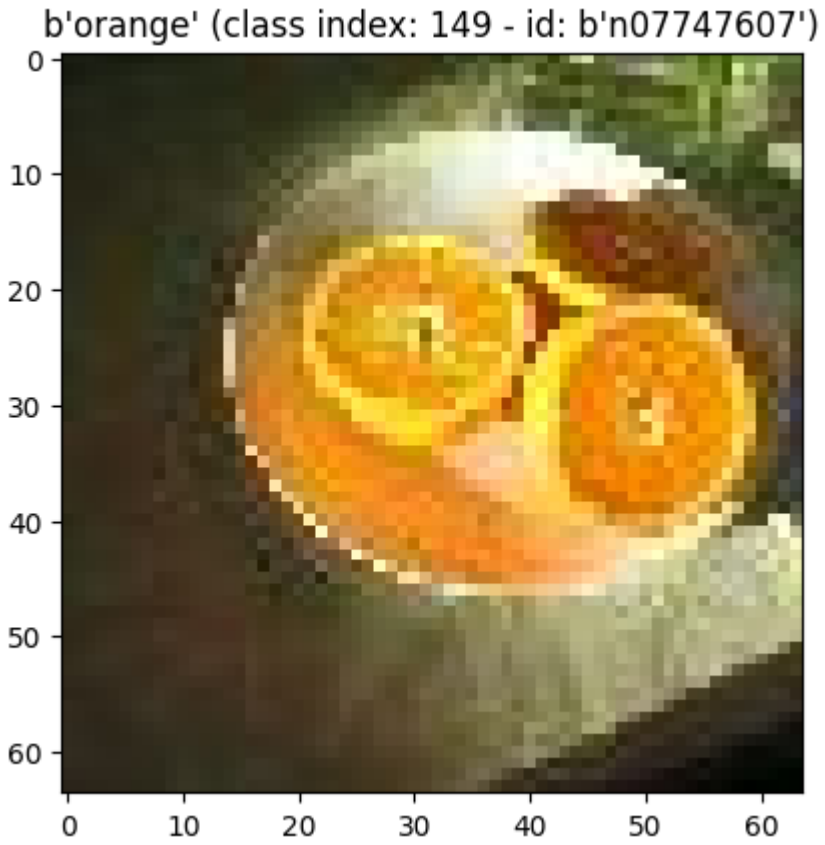
In [3]:
```
# We need to read the "human readable" labels so we can translate with th
# Read the labels file (words.txt)
with open(os.path.abspath('wnids.txt'), 'r') as f:
    wnids = [x.strip() for x in f]

# Map wnids to integer labels
wnid_to_label = {wnid: i for i, wnid in enumerate(wnids)}
label_to_wnid = {v: k for k, v in wnid_to_label.items()}

# Use words.txt to get names for each class
with open(os.path.abspath('words.txt'), 'r') as f:
    wnid_to_words = dict(line.split('\t') for line in f)
    for wnid, words in wnid_to_words.items():
        wnid_to_words[wnid] = [w.strip() for w in words.split(',')]

class_names = [str(wnid_to_words[wnid]) for wnid in wnids]
```

In [4]:
```
# Helper function to get the label name
def img_class(img_data, idx=None):
    image, label, id, label_name = img_data["image"], img_data["label"],
    # Handle batches of images correctly
    if idx != None:
        image, label, id, label_name = img_data["image"][idx], img_data["

    return f"{label_name} (class index: {label} - id: {id})"


# Helper function to show basic info about an image
def img_info(img, idx=None, display=True, title_apend=""):
    image = img['image']

    # Print the class
    class_str = img_class(img, idx)
    print(f"Label: {class_str}")

    # Display the image
    if display:
        plt.figure()
        plt.title(title_apend + class_str)
        # Handle batches correctly
        if image.shape.ndims > 3:
            plt.imshow(image.numpy().reshape(64, 64, 3))
        else:
            plt.imshow(image.numpy())
```

In [5]:
```
# Print the dataset types and info
print("--- Train & Validation dataset info ---")
print(f"Train: {ds_train}")
print(f"Validation: {ds_val}")
# print(f"Dataset Info: {ds_info}") # Uncomment to print Dataset info
```

```
print("\n--- Show an example image ---")
for example in ds_val.take(1):
    img_info(example)

print("\n Show some other examples")
tfds.show_examples(ds_val, ds_info, rows=3, cols=3)
```

```
--- Train & Validation dataset info ---
Train: <_PrefetchDataset element_spec={'id': TensorSpec(shape=(), dtype=tf
.string, name=None), 'image': TensorSpec(shape=(64, 64, 3), dtype=tf.uint8
, name=None), 'label': TensorSpec(shape=(), dtype=tf.int64, name=None), 'm
etadata': {'label_name': TensorSpec(shape=(), dtype=tf.string, name=Non
e)}}>
Validation: <_PrefetchDataset element_spec={'id': TensorSpec(shape=(), dty
pe=tf.string, name=None), 'image': TensorSpec(shape=(64, 64, 3), dtype=tf.
uint8, name=None), 'label': TensorSpec(shape=(), dtype=tf.int64, name=Non
e), 'metadata': {'label_name': TensorSpec(shape=(), dtype=tf.string, name=
None)}}>

--- Show an example image ---
Label: b'orange' (class index: 149 - id: b'n07747607')

 Show some other examples
2025-09-08 12:29:59.539604: W tensorflow/core/kernels/data/cache_dataset_o
ps.cc:858] The calling iterator did not fully read the dataset being cache
d. In order to avoid unexpected truncation of the dataset, the partially c
ached contents of the dataset  will be discarded. This can happen if you h
ave an input pipeline similar to `dataset.cache().take(k).repeat()`. You s
hould use `dataset.take(k).cache().repeat()` instead.
2025-09-08 12:29:59.559024: W tensorflow/core/framework/local_rendezvous.c
c:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequ
ence
2025-09-08 12:29:59.643395: W tensorflow/core/kernels/data/cache_dataset_o
ps.cc:858] The calling iterator did not fully read the dataset being cache
d. In order to avoid unexpected truncation of the dataset, the partially c
ached contents of the dataset  will be discarded. This can happen if you h
ave an input pipeline similar to `dataset.cache().take(k).repeat()`. You s
hould use `dataset.take(k).cache().repeat()` instead.
2025-09-08 12:29:59.654259: W tensorflow/core/framework/local_rendezvous.c
c:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequ
ence
```


b'orange' (class index: 149 - id: b'n07747607')
0
10
20
30
40
50
60
0
10
20
30
40
50
60

140 (140)

60 (60)

153 (153)

150 (150)

9 (9)

116 (116)

81 (81)

119 (119)

185 (185)

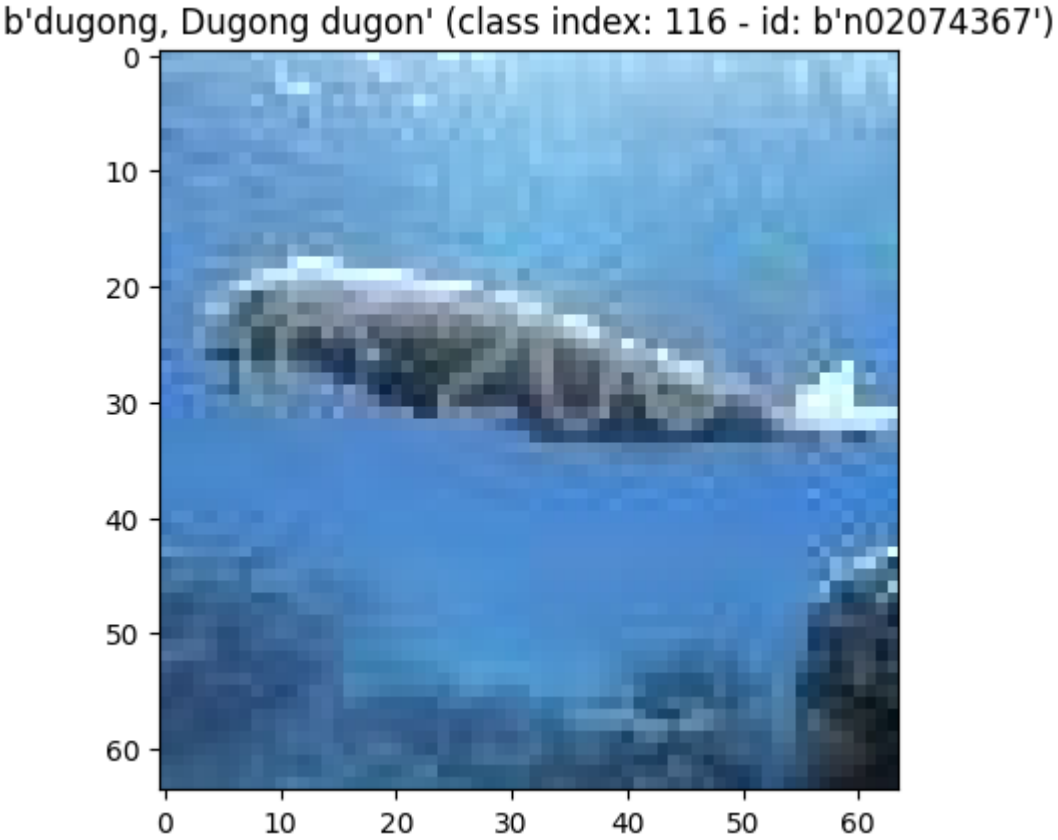
Out[5]:

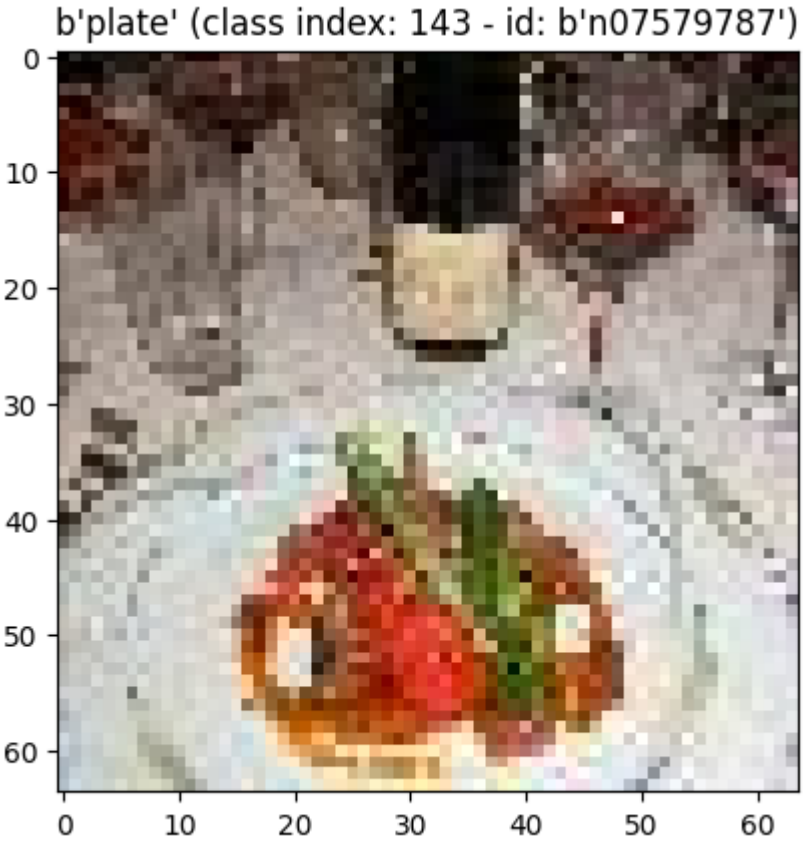
In [6]:

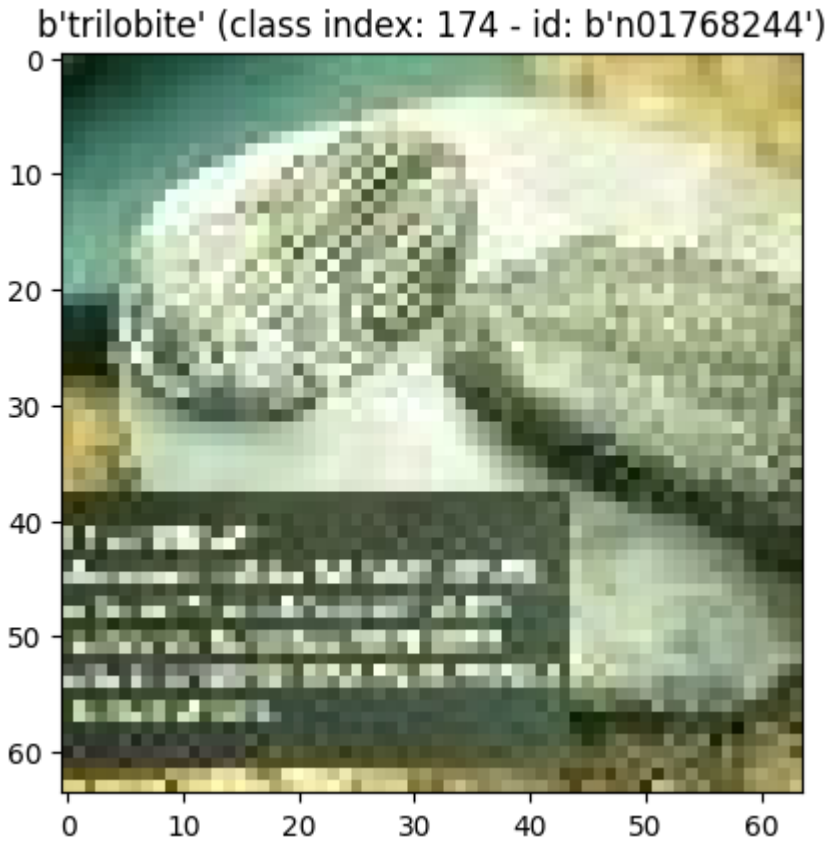
```
# TODO: Print and visualize three inputs from the validation set
#     : Print the stroage data type
#     : Print and note the dimensions of each image
#     : Print the memory required to store each image

# Sample Images
sample_imgs = []
for index, img_data in enumerate(ds_val.take(3)):
    sample_imgs.append(img_data)
    image, label, id, label_name = img_data["image"], img_data["label"],

    print(f'\n--- Image {index} ---')
    # TODO: Your Code Here
    # See example usage: https://github.com/duweisu/tiny-imagenet-tfds
    img_info(img_data) # Print name of image as well as visualization
    #print(f"{image}")
    print(f"Image Shape - {image.shape}") # All images appear to be 3 dim
    print(f"Image Dimensions - {len(image.shape)}")
    print(f"Image Storage - {image.dtype}")
    print(f"Label Index - {label.numpy()}")
    print(f"Id - {id.numpy()}")
    print(f"Size - {image.shape[0] * image.shape[1] * image.shape[2]} Byt
```

```
--- Image 0 ---
Label: b'dugong, Dugong dugon' (class index: 116 - id: b'n02074367')
Image Shape - (64, 64, 3)
Image Dimensions - 3
Image Storage - <dtype: 'uint8'>
Label Index - 116
Id - b'n02074367'
Size - 12288 Bytes

--- Image 1 ---
Label: b'plate' (class index: 143 - id: b'n07579787')
Image Shape - (64, 64, 3)
Image Dimensions - 3
Image Storage - <dtype: 'uint8'>
Label Index - 143
Id - b'n07579787'
Size - 12288 Bytes

--- Image 2 ---
Label: b'trilobite' (class index: 174 - id: b'n01768244')
Image Shape - (64, 64, 3)
Image Dimensions - 3
Image Storage - <dtype: 'uint8'>
Label Index - 174
Id - b'n01768244'
Size - 12288 Bytes
```

```
2025-09-08 12:30:00.506068: W tensorflow/core/kernels/data/cache_dataset_o
ps.cc:858] The calling iterator did not fully read the dataset being cache
d. In order to avoid unexpected truncation of the dataset, the partially c
ached contents of the dataset  will be discarded. This can happen if you h
ave an input pipeline similar to `dataset.cache().take(k).repeat()`. You s
hould use `dataset.take(k).cache().repeat()` instead.
2025-09-08 12:30:00.513842: W tensorflow/core/framework/local_rendezvous.c
c:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequ
ence
```


b'dugong, Dugong dugon' (class index: 116 - id: b'n02074367')
0
10
20
30
40
50
60
0
10
20
30
40
50
60


b'plate' (class index: 143 - id: b'n07579787')
0
10
20
30
40
50
60
0
10
20
30
40
50
60



In [7]:
```
# TODO: Export each of the three inputs to a binary file which will be us
# NOTE: First flatten the array (ex: 4D --> 1D). So 64*64*3 = 12288 eleme

# Make a directory for our image data
img_dir = os.path.abspath('img_data')
pathlib.Path(img_dir).mkdir(exist_ok=True)

# Create a metadata file
with open(os.path.join(img_dir, 'metadata.txt'), 'w') as metadata_file:
    metadata_file.write('Number\t\tDims\t\tClass Data\n')

for index, img_data in enumerate(sample_imgs):

    image, label, id, label_name = img_data["image"], img_data["label"],
    img_flat = image.numpy().flatten()

    # print index, shape and flattened length
    print(f"Image {index} shape: {image.shape}, flattened length: {len(im

    # 'wb' is write as a binary file
    with open(os.path.join(img_dir, f'image_{index}.bin'), 'wb') as img_f
        img_flat.tofile(img_file)

    # 'a' mode in python is appending
    with open(os.path.join(img_dir, 'metadata.txt'), 'a') as metadata_fil
        metadata_file.write(f'{index}\t\t{image.shape}\t\t{label_name}\n'
```

```
Image 0 shape: (64, 64, 3), flattened length: 12288
Image 1 shape: (64, 64, 3), flattened length: 12288
Image 2 shape: (64, 64, 3), flattened length: 12288
```

Model Loading and Inference

In [8]:
```
# TODO: Load the model
# Now we will load the H5 model! Please make sure the h5 model file is pr
# You can download this from the Canvas Page and place it in the same dir

# model_path = os.path.abspath(""/home/<NETID>/path/to/your/lab1/CNN_Tiny
model_path = os.path.abspath("CNN_TinyImageNet.h5")

# TODO: Your Code Here
model = tf.keras.models.load_model(model_path)
model.summary()
```

```
/home/tbibus/CprE587/lab01/cpre-487-587-lab1/lab1_venv/lib64/python3.9/sit
e-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: D
o not pass an `input_shape`/`input_dim` argument to a layer. When using Se
quential models, prefer using an `Input(shape)` object as the first layer
in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/tbibus/CprE587/lab01/cpre-487-587-lab1/lab1_venv/lib64/python3.9/sit
e-packages/keras/src/optimizers/base_optimizer.py:86: UserWarning: Argumen
t `decay` is no longer supported and will be ignored.
  warnings.warn(
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet
to be built. `model.compile_metrics` will be empty until you train or eval
uate the model.
WARNING:absl:Error in loading the saved optimizer state. As a result, your
model is starting with a freshly initialized optimizer.
```

Model: "sequential"

Layer (type)	Output Shape	P
conv2d (Conv2D)	(None, 60, 60, 32)	
conv2d_1 (Conv2D)	(None, 56, 56, 32)	
max_pooling2d (MaxPooling2D)	(None, 28, 28, 32)	
conv2d_2 (Conv2D)	(None, 26, 26, 64)	
conv2d_3 (Conv2D)	(None, 24, 24, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 12, 12, 64)	
conv2d_4 (Conv2D)	(None, 10, 10, 64)	
conv2d_5 (Conv2D)	(None, 8, 8, 128)	
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	
flatten (Flatten)	(None, 2048)	
dense (Dense)	(None, 256)	5
dense_1 (Dense)	(None, 200)	

Total params: 770,218 (2.94 MB)

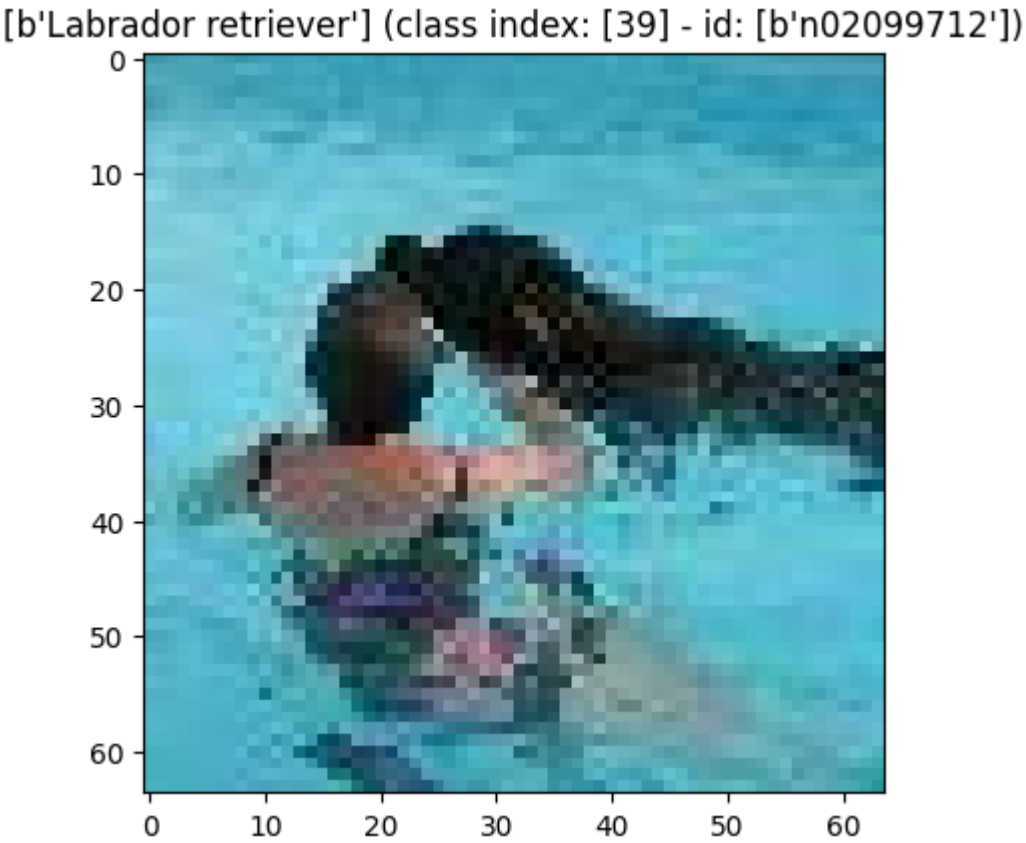
Trainable params: 770,216 (2.94 MB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 2 (12.00 B)

In [9]:
```
# Running infrence on our model
# We can run an infrence of our model by doing the following (we are doin
for example in ds_train.batch(1).take(1):
    img_info(example)

    # Make a prediction
    pred = model.predict(tf.cast(example["image"], tf.float32)/255.0)
    # print(f'Raw 200 Class Weighted Prediction:\n{pred}') # Uncomment to

    # What is out best guess?
    best_guess = tf.math.argmax(pred, axis=1).numpy() # Our output is 200
    print(f'Best Guess [class index]: {class_names[best_guess[0]]} [{best
    print(f'Best Guess Confidence (percent / 1.0): {pred[0][best_guess]}'

    # What are our top 15 guesses?
    top_15 = tf.math.top_k(pred, k=15)
    print(f'Top 15 Guesses (class index): {[f"{class_names[idx]} [{idx}]"
    print(f'Top 15 Guesses Confidence (percent / 1.0): {top_15.values}')
```

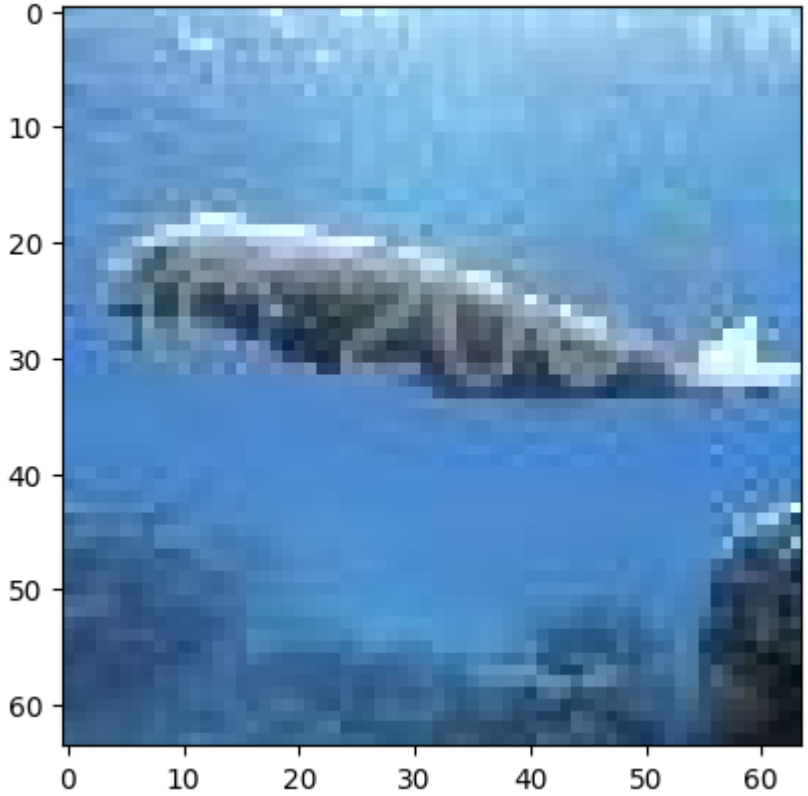
```
2025-09-08 12:30:01.505222: W tensorflow/core/kernels/data/cache_dataset_o
ps.cc:858] The calling iterator did not fully read the dataset being cache
d. In order to avoid unexpected truncation of the dataset, the partially c
ached contents of the dataset  will be discarded. This can happen if you h
ave an input pipeline similar to `dataset.cache().take(k).repeat()`. You s
hould use `dataset.take(k).cache().repeat()` instead.
Label: [b'Labrador retriever'] (class index: [39] - id: [b'n02099712'])
WARNING: All log messages before absl::InitializeLog() is called are writt
en to STDERR
I0000 00:00:1757352601.633373  128766 service.cc:145] XLA service 0x7f7f70
007630 initialized for platform CUDA (this does not guarantee that XLA wil
l be used). Devices:
I0000 00:00:1757352601.633407  128766 service.cc:153]   StreamExecutor dev
ice (0): NVIDIA A40-8Q, Compute Capability 8.6
2025-09-08 12:30:01.639562: I tensorflow/compiler/mlir/tensorflow/utils/du
mp_mlir_util.cc:268] disabling MLIR crash reproducer, set env var `MLIR_CR
ASH_REPRODUCER_DIRECTORY` to enable.
2025-09-08 12:30:01.682660: I external/local_xla/xla/stream_executor/cuda/
cuda_dnn.cc:465] Loaded cuDNN version 8907
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 565ms/step
Best Guess [class index]: ['snorkel'] [111]
Best Guess Confidence (percent / 1.0): [0.56190544]
Top 15 Guesses (class index): ["['snorkel'] [111]", "['black stork', 'Cico
nia nigra'] [115]", "['albatross', 'mollymawk'] [35]", "['goose'] [67]",
"['binoculars', 'field glasses', 'opera glasses'] [197]", "['dumbbell'] [1
54]", "['magnetic compass'] [191]", "['syringe'] [162]", "['snail'] [33]",
"['remote control', 'remote'] [54]", "['swimming trunks', 'bathing trunk
s'] [136]", "['bucket', 'pail'] [190]", "['bikini', 'two-piece'] [142]",
"['gasmask', 'respirator', 'gas helmet'] [83]", "['acorn'] [195]"]
Top 15 Guesses Confidence (percent / 1.0): [[0.56190544 0.07275921 0.03228
819 0.031647   0.02180277 0.02139886
  0.01891495 0.01872069 0.00884652 0.0081669  0.00779579 0.00758822
  0.00681273 0.00673694 0.00657327]]
I0000 00:00:1757352602.095536  128766 device_compiler.h:188] Compiled clus
ter using XLA!  This line is logged at most once for the lifetime of the p
rocess.
2025-09-08 12:30:02.134232: W tensorflow/core/framework/local_rendezvous.c
c:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequ
ence
```



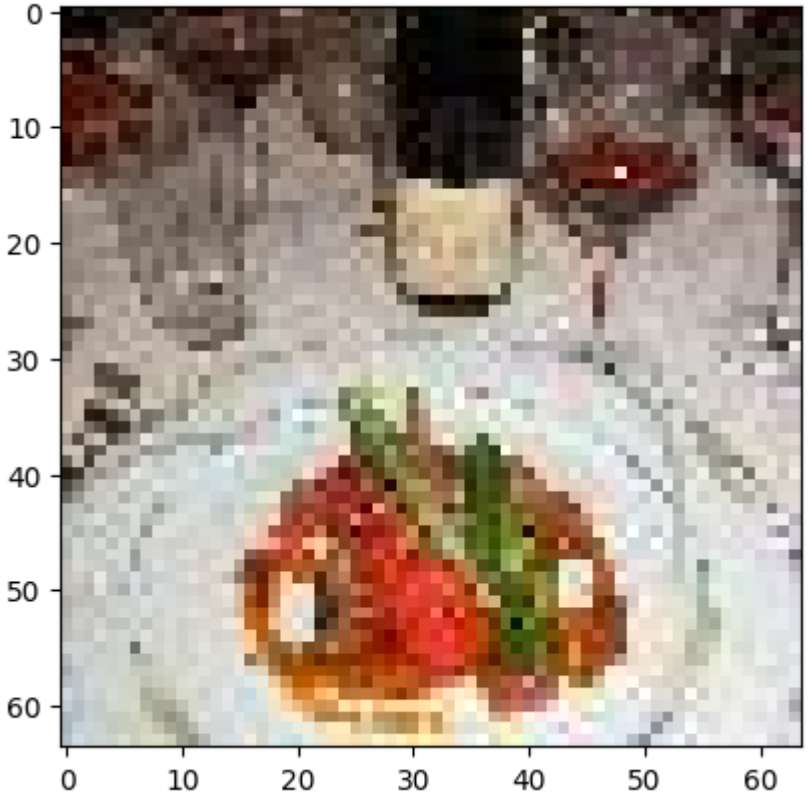
In [10]:
```
# TODO: Run infrence for our previous 3 sample images

# TODO: Your Code Here


for index, img_data in enumerate(sample_imgs):
    print(f"------------- Image {index+1} ----")
    img_info(img_data)
    img = sample_imgs[index]["image"]
    normalized = tf.cast(img, tf.float32) / 255.0
    normalized = tf.expand_dims(normalized, axis=0)
    pred = model.predict(normalized)

    # What is out best guess?
    best_guess = tf.math.argmax(pred, axis=1).numpy() # Our output is 200
    print(f'Best Guess [class index]: {class_names[best_guess[0]]} [{best
    print(f'Best Guess Confidence (percent / 1.0): {pred[0][best_guess]}'

    # What are our top 15 guesses?
    top_15 = tf.math.top_k(pred, k=15)
    print(f'Top 15 Guesses (class index): {[f"{class_names[idx]} [{idx}]"
    print(f'Top 15 Guesses Confidence (percent / 1.0): {top_15.values}')
```

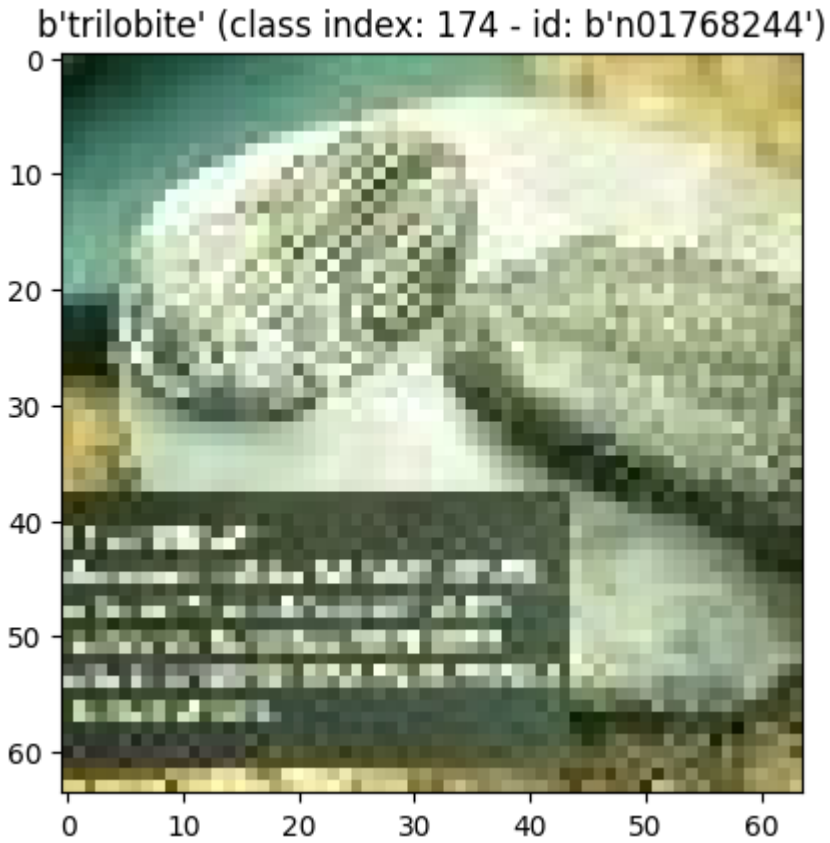
```
------------- Image 1 ----
Label: b'dugong, Dugong dugon' (class index: 116 - id: b'n02074367')
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step
Best Guess [class index]: ['alp'] [60]
Best Guess Confidence (percent / 1.0): [0.51806486]
Top 15 Guesses (class index): ["['alp'] [60]", "['American alligator', 'Al
ligator mississipiensis'] [160]", "['crane'] [103]", "['black stork', 'Cic
onia nigra'] [115]", "['snorkel'] [111]", "['dugong', 'Dugong dugon'] [11
6]", "['syringe'] [162]", "['centipede'] [161]", "['albatross', 'mollymaw
k'] [35]", "['steel arch bridge'] [130]", "['sea cucumber', 'holothurian']
[37]", "['lakeside', 'lakeshore'] [56]", "['mantis', 'mantid'] [51]", "['b
lack widow', 'Latrodectus mactans'] [141]", "['chain'] [171]"]
Top 15 Guesses Confidence (percent / 1.0): [[0.51806486 0.16286987 0.07291
592 0.03848748 0.03651674 0.01611376
  0.01471755 0.01080083 0.00993178 0.00951513 0.00920609 0.00855567
  0.00729422 0.00716135 0.00714192]]
------------- Image 2 ----
Label: b'plate' (class index: 143 - id: b'n07579787')
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step
Best Guess [class index]: ['plate'] [143]
Best Guess Confidence (percent / 1.0): [0.08763183]
Top 15 Guesses (class index): ["['plate'] [143]", "['comic book'] [173]",
"['ice cream', 'icecream'] [28]", "['mashed potato'] [167]", "['guacamol
e'] [53]", "['sewing machine'] [129]", "['American lobster', 'Northern lob
ster', 'Maine lobster', 'Homarus americanus'] [151]", "['kimono'] [172]",
"['teddy', 'teddy bear'] [144]", "['teapot'] [71]", "['cauliflower'] [19
8]", "['pomegranate'] [170]", '["potter\'s wheel"] [96]', "['pizza', 'pizz
a pie'] [16]", "['refrigerator', 'icebox'] [132]"]
Top 15 Guesses Confidence (percent / 1.0): [[0.08763183 0.06760038 0.06497
597 0.05135423 0.04188608 0.04088187
  0.03957929 0.03659131 0.02601212 0.02580988 0.02221835 0.0203972
  0.01793958 0.01784339 0.01743805]]
------------- Image 3 ----
Label: b'trilobite' (class index: 174 - id: b'n01768244')
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step
Best Guess [class index]: ['bannister', 'banister', 'balustrade', 'baluste
rs', 'handrail'] [189]
Best Guess Confidence (percent / 1.0): [0.11206224]
Top 15 Guesses (class index): ["['bannister', 'banister', 'balustrade', 'b
alusters', 'handrail'] [189]", "['umbrella'] [18]", "['cash machine', 'cas
h dispenser', 'automated teller machine', 'automatic teller machine', 'aut
omated teller', 'automatic teller', 'ATM'] [106]", "['stopwatch', 'stop wa
tch'] [137]", "['CD player'] [70]", "['reel'] [1]", "['parking meter'] [1
0]", "['birdhouse'] [72]", "['remote control', 'remote'] [54]", "['project
ile', 'missile'] [49]", "['cannon'] [80]", "['nail'] [29]", "['thatch', 't
hatched roof'] [139]", "['chain'] [171]", "['magnetic compass'] [191]"]
Top 15 Guesses Confidence (percent / 1.0): [[0.11206224 0.10292937 0.07692
856 0.07312399 0.0366116  0.03567372
  0.03156932 0.0304611  0.02605739 0.02331767 0.02210626 0.02074449
  0.01924161 0.01602937 0.01550021]]
```

b'dugong, Dugong dugon' (class index: 116 - id: b'n02074367')



b'plate' (class index: 143 - id: b'n07579787')





In [11]:
```
# TODO: Calculate the Top-1, Top-5, and Top-10 Accuracy of the validation
total = acc_top1 = acc_top5 = acc_top10 = 0
top1_correct = top5_correct = top10_correct = total_images = 0

total_images = 0

batch_size = 256

#ds_val = ds_val.batch(batch_size)
#ds_val = ds_val.prefetch(tf.data.AUTOTUNE)

# TODO: Your Code Here
for batch in ds_val.batch(batch_size).take(10000 // batch_size + 1):
    if(total_images >= 10000):
        break

    #print(f"images analyzed: {total_images}\n")
    images = batch["image"]
    labels = batch["label"]
    images = tf.cast(images, tf.float32) / 255.0

    #images = tf.expand_dims(images, axis=0)

    #pred = model.predict(tf.cast(example["image"], tf.float32)/255.0)
    # Make a prediction
    pred = model.predict(images, verbose=0)

    # Convert labels to int32 if one-hot encoded
    if len(labels.shape) > 1:  # Check if labels are one-hot
        labels = tf.argmax(labels, axis=1)
    labels = tf.cast(labels, tf.int32)  # Shape: (batch_size,)

    labels = tf.expand_dims(labels, axis=1)  # Shape: (batch_size, 1)
```

```
    #get correct counts

    top10 = tf.math.top_k(pred, k=10)
    top10_indices = top10.indices

    top1_matches = tf.reduce_any(tf.equal(labels, top10_indices[:, :1]))
    top1_correct += tf.reduce_sum(tf.cast(top1_matches, tf.int32)).numpy(

    # Top-5 accuracy: True label in top-5 predictions
    top5_matches = tf.reduce_any(tf.equal(labels, top10_indices[:, :5]),
    top5_correct += tf.reduce_sum(tf.cast(top5_matches, tf.int32)).numpy(

    # Top-10 accuracy: True label in top-10 predictions
    top10_matches = tf.reduce_any(tf.equal(labels, top10_indices), axis=1
    top10_correct += tf.reduce_sum(tf.cast(top10_matches, tf.int32)).nump

    # Update total images
    total_images += batch_size

print(f"Total Images: {total_images}\n")
acc_top1 = top1_correct / total_images
acc_top5 = top5_correct / total_images
acc_top10 = top10_correct / total_images

total = (acc_top1 + acc_top5 + acc_top10) / 3.0

print(f"Top 1 Accuracy : {acc_top1 * 100.0}\n")
print(f"Top 5 Accuracy : {acc_top5 * 100.0}\n")
print(f"Top 10 Accuracy : {acc_top10 * 100.0}\n")
print(f"Total Accuracy : {total * 100.0}\n")
```

```
WARNING: All log messages before absl::InitializeLog() is called are writt
en to STDERR
I0000 00:00:1757352606.853796  130004 asm_compiler.cc:369] ptxas warning :
Registers are spilled to local memory in function 'triton_gemm_dot_93', 16
bytes spill stores, 16 bytes spill loads

Total Images: 10240

Top 1 Accuracy : 0.390625

Top 5 Accuracy : 48.251953125

Top 10 Accuracy : 60.29296875

Total Accuracy : 36.31184895833333

2025-09-08 12:30:07.406203: W tensorflow/core/framework/local_rendezvous.c
c:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequ
ence
```

In [12]:
```
# TODO: Print all of the possible classes of the dataset

# TODO: Your Code Here
```

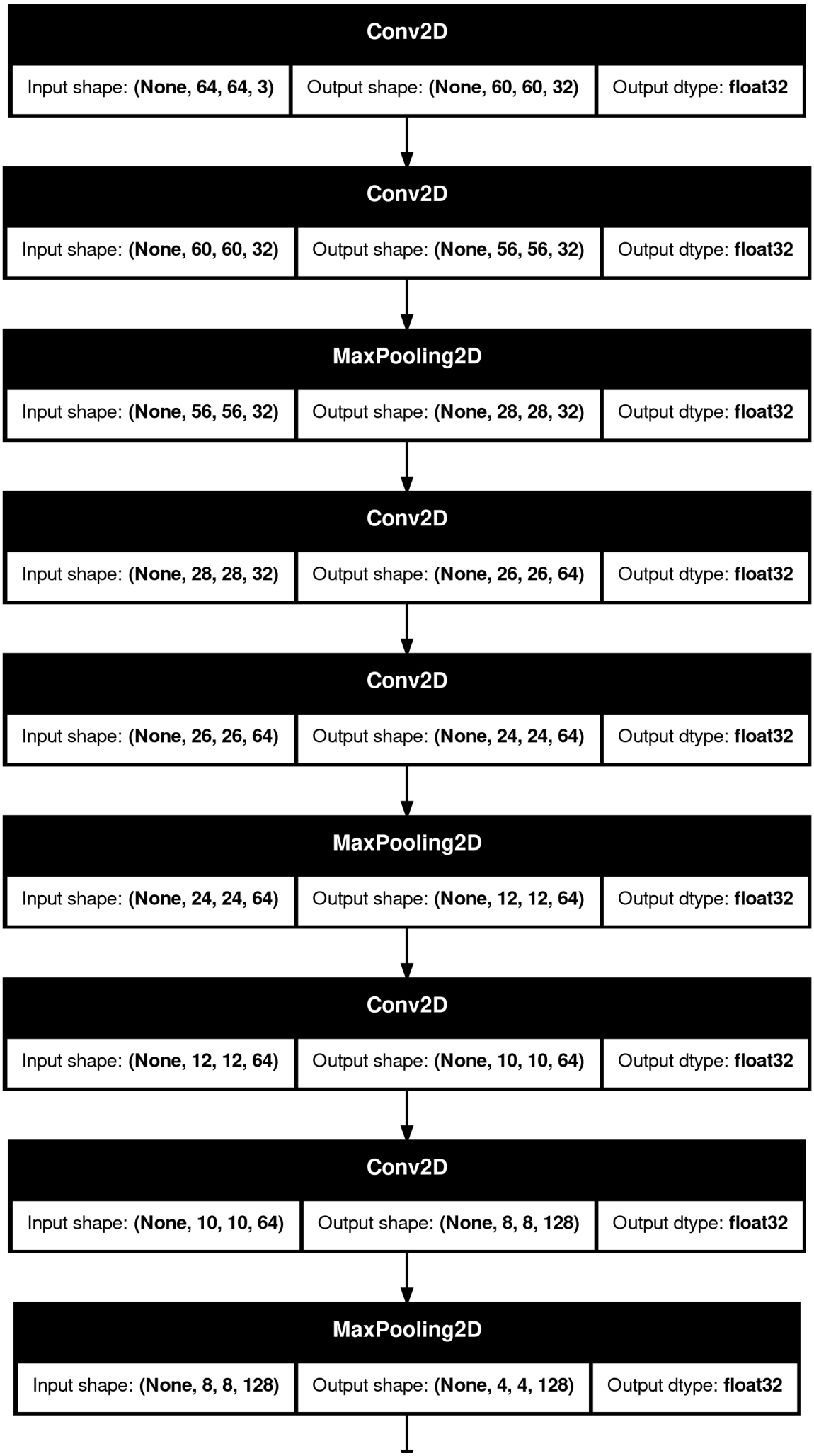
```
train_labels = ds_train.map(lambda x: x['label']).batch(len(ds_train)).as

print("classes in training dataset:\n")
print(np.unique(train_labels))

val_labels = ds_val.map(lambda x: x['label']).batch(len(ds_val)).as_numpy

print("\nclasses in validation dataset:\n")
print(np.unique(val_labels))
```

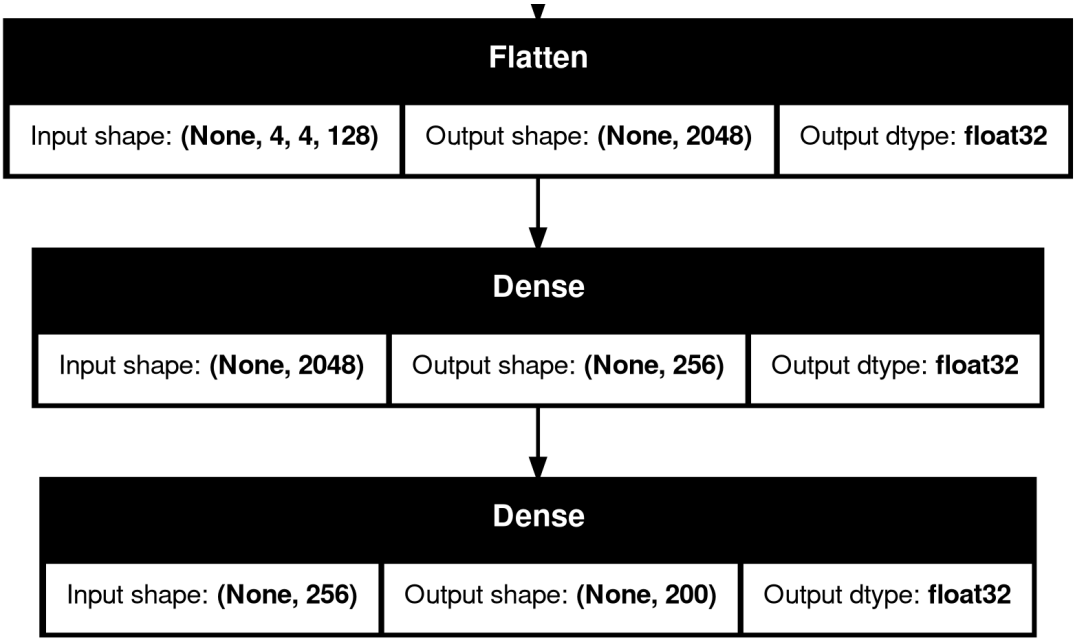
```
classes in training dataset:

[  0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17
  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35
  36  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53
  54  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71
  72  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89
  90  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107
 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125
 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143
 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161
 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179
 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197
 198 199]

classes in validation dataset:

[  0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17
  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35
  36  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53
  54  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71
  72  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89
  90  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107
 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125
 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143
 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161
 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179
 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197
 198 199]
```

Model Exploration

In [13]:
```
# TODO: Visualize the model in Netron (https://netron.app/) and include a
tf.keras.utils.plot_model(model, "model.png", show_shapes=True, show_dtyp
```

Out[13]:

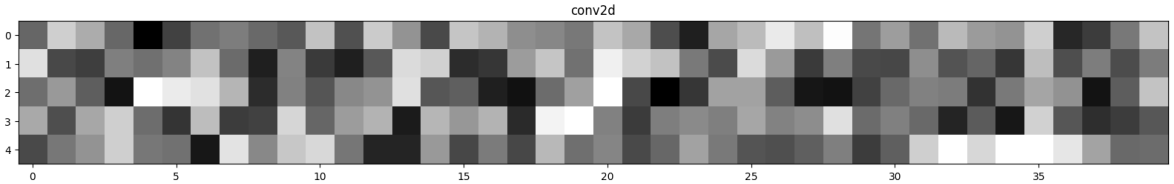

Conv2D
Input shape: (None, 64, 64, 3)
Output shape: (None, 60, 60, 32)
Output dtype: float32
Conv2D
Input shape: (None, 60, 60, 32)
Output shape: (None, 56, 56, 32)
Output dtype: float32
MaxPooling2D
Input shape: (None, 56, 56, 32)
Output shape: (None, 28, 28, 32)
Output dtype: float32
Conv2D
Input shape: (None, 28, 28, 32)
Output shape: (None, 26, 26, 64)
Output dtype: float32
Conv2D
Input shape: (None, 26, 26, 64)
Output shape: (None, 24, 24, 64)
Output dtype: float32
MaxPooling2D
Input shape: (None, 24, 24, 64)
Output shape: (None, 12, 12, 64)
Output dtype: float32
Conv2D
Input shape: (None, 12, 12, 64)
Output shape: (None, 10, 10, 64)
Output dtype: float32
Conv2D
Input shape: (None, 10, 10, 64)
Output shape: (None, 8, 8, 128)
Output dtype: float32
MaxPooling2D
Input shape: (None, 8, 8, 128)
Output shape: (None, 4, 4, 128)
Output dtype: float32

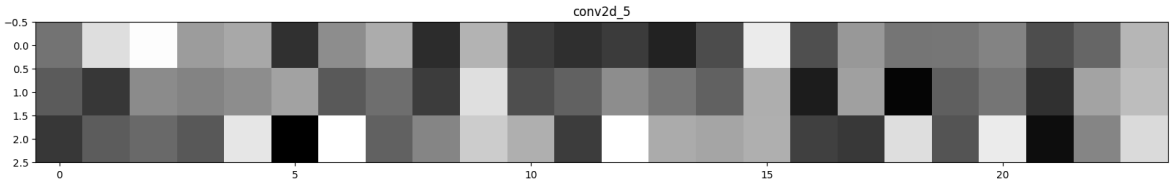


In [14]:
```
# We can view the layer weights. Here we consider them as images of featu
# TODO: Visualize the 2 convolutional layers filter sets (weights) (one a

# TODO: Your Code Here

layers = [layer for layer in model.layers]

first_layer, last_layer = layers[0], layers[-5] # first and last conv lay
first_weights, last_weights = first_layer.get_weights()[0], last_layer.ge

# vosializes the kernels in a belt form
def visualize_kernels(weights, layer_name):
    height, width, in_channels, out_channels = weights.shape # for weight
    k = min(out_channels, 8) # min 8 increases the size for easier readab
    size = height

    # we need an empty belt (getting this right took forever)
    image_belt = np.zeros((size, size * k), dtype=np.uint8) # Image belt

    # # from alayticsvidhya article
    for i in range(k):
        feature_image = np.mean(weights[:, :, :, i], axis=2) # axis 2 has
        feature_image -= feature_image.mean()
        feature_image /= feature_image.std()
        feature_image *= 64
        feature_image += 128
        feature_image = np.clip(feature_image, 0, 255).astype('uint8')
        image_belt[:, i * size : (i + 1) * size] = feature_image

    # Print belt (from analyticsvidya article)
    scale = 20. / k
    plt.figure(figsize=(scale * k, scale))
    plt.title(layer_name)
    plt.grid(False)
    plt.imshow(image_belt, cmap='gray', aspect='auto')

# do both layers
visualize_kernels(first_weights, first_layer.name)
visualize_kernels(last_weights, last_layer.name)
```

```
#   print weigt details
def print_weight_details(weights, layer_name):
    dtype, shape = weights.dtype, weights.shape
    num_params = np.prod(shape)
    memory_bytes = num_params * 4
    print(f"{layer_name}: {dtype}, Shape: {shape}, Memory: {memory_bytes

print_weight_details(first_weights, first_layer.name)
print_weight_details(last_weights, last_layer.name)
```

```
conv2d: float32, Shape: (5, 5, 3, 32), Memory: 9.38 KB
conv2d_5: float32, Shape: (3, 3, 64, 128), Memory: 288.00 KB
```



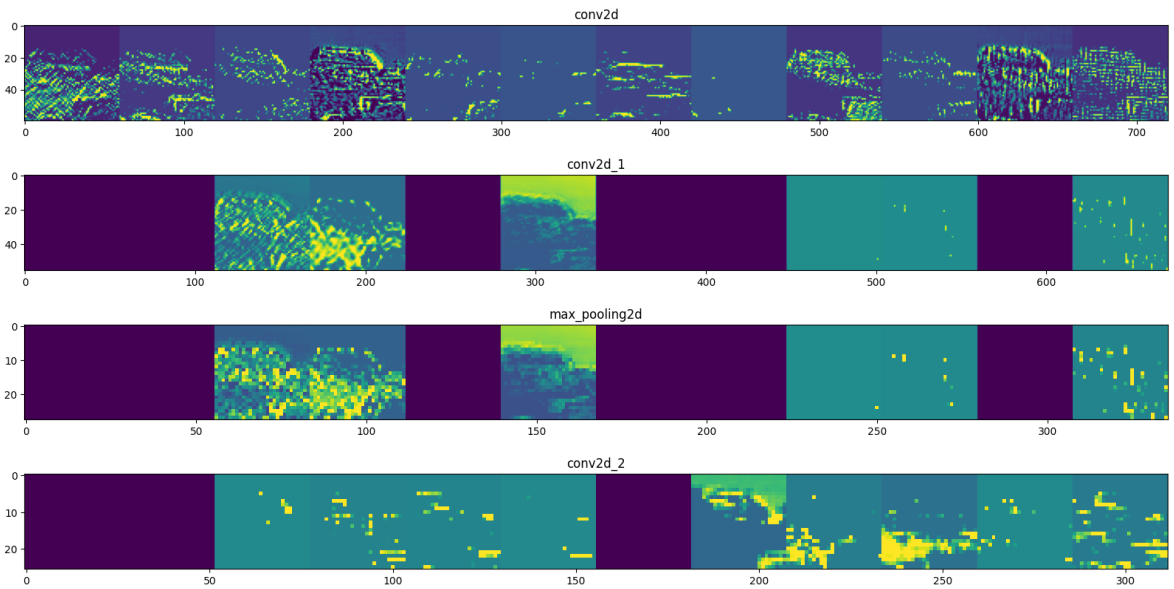


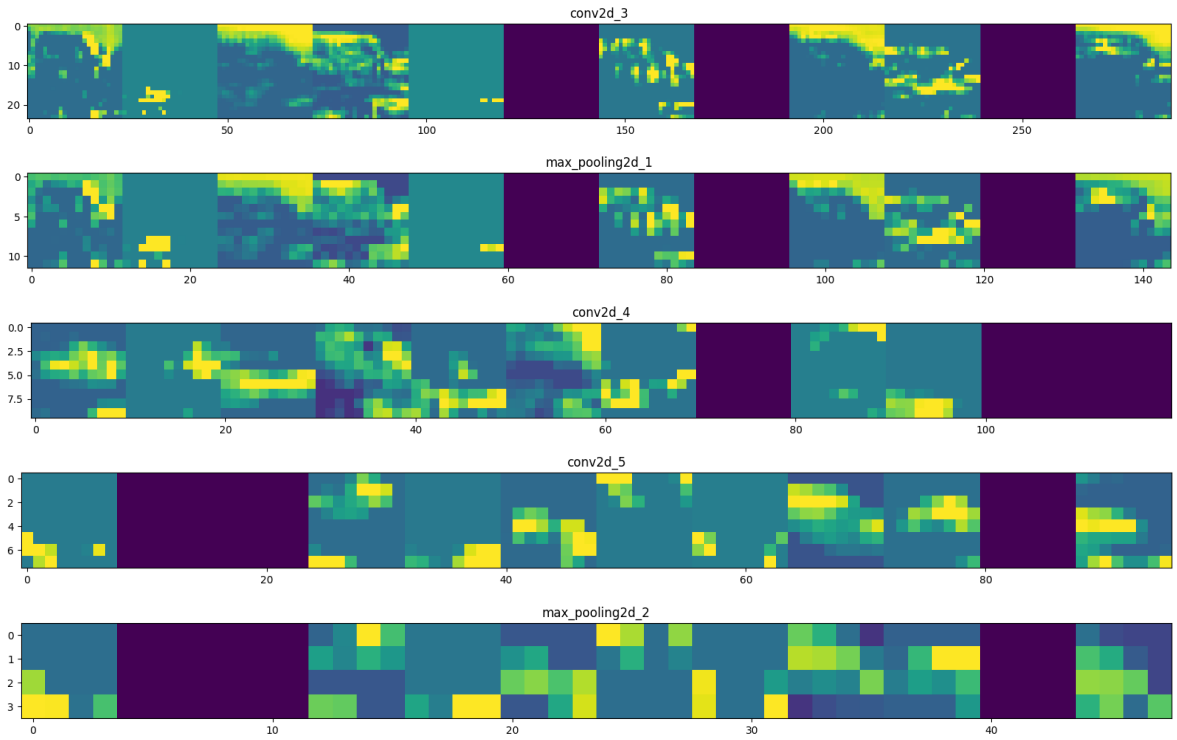
In [15]:
```
# We can view the layer outputs as well. Here we consider them as images
# TODO: Visualize the 2 convolutional layers outputs (intermediate featur

# TODO: Your Code Here
def visualize_features(layer_name, feature_map):
    for layer_name, feature_map in zip(layer_names, feature_maps): # Numb
        if len(feature_map.shape) != 4:
            return
        k = min(feature_map.shape[-1], 12)  # Limit to 12 for readability
        size=feature_map.shape[1]
        image_belt = np.zeros((size, size * k), dtype=np.uint8)

        #iterating over a feature map of a particular layer to separate a
        for i in range(k):
            feature_image = feature_map[0, :, :, i]
            feature_image-= feature_image.mean()
            feature_image/= feature_image.std ()
            feature_image*=  64
            feature_image+= 128
            feature_image= np.clip(feature_image, 0, 255).astype('uint8')
            image_belt[:, i * size : (i + 1) * size] = feature_image

        scale = 20. / k
        plt.figure( figsize=(scale * k, scale) )
        plt.title ( layer_name )
        plt.grid  ( False )
        plt.imshow( image_belt, aspect='auto')


# get an image from ds_val


layer_names = [layer.name for layer in model.layers]
layer_outputs = [layer.output for layer in model.layers]
```

```
feature_map_model = tf.keras.models.Model(inputs=model.inputs, outputs=la

for batch in ds_val.batch(1).take(1):
    input_img = batch["image"]
    input_img = tf.cast(input_img, tf.float32) / 255.0
    feature_maps = feature_map_model.predict(input_img)
    visualize_features(layer_names[4], feature_maps[4])

#   print weigt details
def print_weight_details(weights, layer_name):
    dtype, shape = weights.dtype, weights.shape
    num_params = np.prod(shape)
    memory_bytes = num_params * 4
    print(f"{layer_name}: {dtype}, Shape: {shape}, Memory: {memory_bytes

for layer in model.layers:
    if len(layer.get_weights()) == 0:
        continue
    weights = layer.get_weights()[0]
    layer_name = layer.name
    print_weight_details(weights, layer_name)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 777ms/step
conv2d: float32, Shape: (5, 5, 3, 32), Memory: 9.38 KB
conv2d_1: float32, Shape: (5, 5, 32, 32), Memory: 100.00 KB
conv2d_2: float32, Shape: (3, 3, 32, 64), Memory: 72.00 KB
conv2d_3: float32, Shape: (3, 3, 64, 64), Memory: 144.00 KB
conv2d_4: float32, Shape: (3, 3, 64, 64), Memory: 144.00 KB
conv2d_5: float32, Shape: (3, 3, 64, 128), Memory: 288.00 KB
dense: float32, Shape: (2048, 256), Memory: 2048.00 KB
dense_1: float32, Shape: (256, 200), Memory: 200.00 KB
```

```
/tmp/ipykernel_128718/3795407857.py:17: RuntimeWarning: invalid value enco
untered in divide
  feature_image/= feature_image.std ()
/tmp/ipykernel_128718/3795407857.py:20: RuntimeWarning: invalid value enco
untered in cast
  feature_image= np.clip(feature_image, 0, 255).astype('uint8')
2025-09-08 12:30:11.191780: W tensorflow/core/framework/local_rendezvous.c
c:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequ
ence
```





In [16]:
```
# TODO: Export the filters/weights se we can use them later
# Make a directory for our image data
model_dir = os.path.abspath('model_data')
pathlib.Path(model_dir).mkdir(exist_ok=True)


# Export each image
conv_index = dense_index = 1 # layer index starts from one
for layer_idx, layer in enumerate(model.layers):
    if re.match(r'(conv|dense)', layer.name):
        weight_file_name = os.path.join(model_dir, f'{layer.name}_weights
        bias_file_name = os.path.join(model_dir, f'{layer.name}_bias.bin'
    else: continue

    assert layer.weights[0].name.endswith('kernel')
    assert layer.weights[1].name.endswith('bias')

    # TODO: Your Code Here
    weights = layer.get_weights()[0]
    biases = layer.get_weights()[1]
    weights_flat = weights.flatten()
    biases_flat = biases.flatten()
    with open(weight_file_name, 'wb') as weight_file:
        weights_flat.tofile(weight_file)
    with open(bias_file_name, 'wb') as bias_file:
        biases_flat.tofile(bias_file)
    print(f"Exported {layer.name} weights to {weight_file_name} and biase
```

```
Exported conv2d weights to /home/tbibus/CprE587/lab01/cpre-487-587-lab1/mo
del_data/conv2d_weights.bin and biases to /home/tbibus/CprE587/lab01/cpr
e-487-587-lab1/model_data/conv2d_bias.bin
Exported conv2d_1 weights to /home/tbibus/CprE587/lab01/cpre-487-587-lab1/
model_data/conv2d_1_weights.bin and biases to /home/tbibus/CprE587/lab01/c
pre-487-587-lab1/model_data/conv2d_1_bias.bin
Exported conv2d_2 weights to /home/tbibus/CprE587/lab01/cpre-487-587-lab1/
model_data/conv2d_2_weights.bin and biases to /home/tbibus/CprE587/lab01/c
pre-487-587-lab1/model_data/conv2d_2_bias.bin
Exported conv2d_3 weights to /home/tbibus/CprE587/lab01/cpre-487-587-lab1/
model_data/conv2d_3_weights.bin and biases to /home/tbibus/CprE587/lab01/c
pre-487-587-lab1/model_data/conv2d_3_bias.bin
Exported conv2d_4 weights to /home/tbibus/CprE587/lab01/cpre-487-587-lab1/
model_data/conv2d_4_weights.bin and biases to /home/tbibus/CprE587/lab01/c
pre-487-587-lab1/model_data/conv2d_4_bias.bin
Exported conv2d_5 weights to /home/tbibus/CprE587/lab01/cpre-487-587-lab1/
model_data/conv2d_5_weights.bin and biases to /home/tbibus/CprE587/lab01/c
pre-487-587-lab1/model_data/conv2d_5_bias.bin
Exported dense weights to /home/tbibus/CprE587/lab01/cpre-487-587-lab1/mod
el_data/dense_weights.bin and biases to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/model_data/dense_bias.bin
Exported dense_1 weights to /home/tbibus/CprE587/lab01/cpre-487-587-lab1/m
odel_data/dense_1_weights.bin and biases to /home/tbibus/CprE587/lab01/cpr
e-487-587-lab1/model_data/dense_1_bias.bin
```

In [17]:

```
# TODO: Export the intermediate layer outputs for each input image for al
img_dir = os.path.abspath('img_data')
pathlib.Path(img_dir).mkdir(exist_ok=True)

# Create a model that outputs all intermediate layers
feature_map_model = tf.keras.models.Model(inputs=model.inputs, outputs=la

for img_idx, img_data in enumerate(sample_imgs):
    file_dir = os.path.join(img_dir, f'test_input_{img_idx}')
    pathlib.Path(file_dir).mkdir(exist_ok=True)

    # prep image
    img = img_data["image"]
    input_img = tf.cast(img, tf.float32) / 255.0
    input_img = tf.expand_dims(input_img, axis=0)

    feature_maps = feature_map_model.predict(input_img)

    #   flatten and export feature maps
    for layer_idx, fmap in enumerate(feature_maps):
        layer_name = layer_names[layer_idx]
        fmap_flat = fmap.flatten()
        out_path = os.path.join(file_dir, f'{layer_name}_output.bin')
        with open(out_path, 'wb') as out_file:
            fmap_flat.tofile(out_file)
        print(f"exported {layer_name} output for image {img_idx} to {out_
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 669ms/step
exported conv2d output for image 0 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_0/conv2d_output.bin
exported conv2d_1 output for image 0 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_0/conv2d_1_output.bin
exported max_pooling2d output for image 0 to /home/tbibus/CprE587/lab01/cp
re-487-587-lab1/img_data/test_input_0/max_pooling2d_output.bin
exported conv2d_2 output for image 0 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_0/conv2d_2_output.bin
exported conv2d_3 output for image 0 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_0/conv2d_3_output.bin
exported max_pooling2d_1 output for image 0 to /home/tbibus/CprE587/lab01/
cpre-487-587-lab1/img_data/test_input_0/max_pooling2d_1_output.bin
exported conv2d_4 output for image 0 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_0/conv2d_4_output.bin
exported conv2d_5 output for image 0 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_0/conv2d_5_output.bin
exported max_pooling2d_2 output for image 0 to /home/tbibus/CprE587/lab01/
cpre-487-587-lab1/img_data/test_input_0/max_pooling2d_2_output.bin
exported flatten output for image 0 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_0/flatten_output.bin
exported dense output for image 0 to /home/tbibus/CprE587/lab01/cpre-487-5
87-lab1/img_data/test_input_0/dense_output.bin
exported dense_1 output for image 0 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_0/dense_1_output.bin
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step
exported conv2d output for image 1 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_1/conv2d_output.bin
exported conv2d_1 output for image 1 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_1/conv2d_1_output.bin
exported max_pooling2d output for image 1 to /home/tbibus/CprE587/lab01/cp
re-487-587-lab1/img_data/test_input_1/max_pooling2d_output.bin
exported conv2d_2 output for image 1 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_1/conv2d_2_output.bin
exported conv2d_3 output for image 1 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_1/conv2d_3_output.bin
exported max_pooling2d_1 output for image 1 to /home/tbibus/CprE587/lab01/
cpre-487-587-lab1/img_data/test_input_1/max_pooling2d_1_output.bin
exported conv2d_4 output for image 1 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_1/conv2d_4_output.bin
exported conv2d_5 output for image 1 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_1/conv2d_5_output.bin
exported max_pooling2d_2 output for image 1 to /home/tbibus/CprE587/lab01/
cpre-487-587-lab1/img_data/test_input_1/max_pooling2d_2_output.bin
exported flatten output for image 1 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_1/flatten_output.bin
exported dense output for image 1 to /home/tbibus/CprE587/lab01/cpre-487-5
87-lab1/img_data/test_input_1/dense_output.bin
exported dense_1 output for image 1 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_1/dense_1_output.bin
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step
exported conv2d output for image 2 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_2/conv2d_output.bin
exported conv2d_1 output for image 2 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_2/conv2d_1_output.bin
exported max_pooling2d output for image 2 to /home/tbibus/CprE587/lab01/cp
re-487-587-lab1/img_data/test_input_2/max_pooling2d_output.bin
exported conv2d_2 output for image 2 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_2/conv2d_2_output.bin
exported conv2d_3 output for image 2 to /home/tbibus/CprE587/lab01/cpre-48
```

```
7-587-lab1/img_data/test_input_2/conv2d_3_output.bin
exported max_pooling2d_1 output for image 2 to /home/tbibus/CprE587/lab01/
cpre-487-587-lab1/img_data/test_input_2/max_pooling2d_1_output.bin
exported conv2d_4 output for image 2 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_2/conv2d_4_output.bin
exported conv2d_5 output for image 2 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_2/conv2d_5_output.bin
exported max_pooling2d_2 output for image 2 to /home/tbibus/CprE587/lab01/
cpre-487-587-lab1/img_data/test_input_2/max_pooling2d_2_output.bin
exported flatten output for image 2 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_2/flatten_output.bin
exported dense output for image 2 to /home/tbibus/CprE587/lab01/cpre-487-5
87-lab1/img_data/test_input_2/dense_output.bin
exported dense_1 output for image 2 to /home/tbibus/CprE587/lab01/cpre-48
7-587-lab1/img_data/test_input_2/dense_1_output.bin
```

Tensorboard

In [18]:
```
# Setup for profiling
tf.profiler.experimental.ProfilerOptions(
    host_tracer_level=1, python_tracer_level=0, device_tracer_level=1
)

log_dir = os.path.abspath(os.path.join('log_data'))
log_dir_run = os.path.abspath(os.path.join(log_dir, datetime.datetime.now
pathlib.Path(log_dir_run).mkdir(exist_ok=True, parents=True)

try:
    tf.profiler.experimental.stop()
except:
    test = 2
finally:
    test = 1
```

In [ ]:
```
# TODO: Sample Profiling - Inference for a single image:

# Define inference step with tf.function for optimization

# I dont know what the hell to do here...
# Issue resolved, was not looking at the toolflow tool dropdown

@tf.function
def inference_step(image):
    return model(image)

tf.profiler.experimental.start(log_dir_run)

for img_data in ds_val.take(3):
    img = img_data["image"]
    img = tf.cast(img, tf.float32) / 255.0
    img = tf.expand_dims(img, axis=0)
    inference_step(img)
    print("image done")

tf.profiler.experimental.stop()
```

In [ ]:
```
# Load the TensorBoard notebook extension.
%load_ext tensorboard
```

```
# Launch TensorBoard and navigate to the Profile tab to view performance
# *** Please note just execute this command once in a session and
# then logs for subsequent runs would be auto detected in tensorboard- ur
print(log_dir_run)
%tensorboard --logdir={log_dir_run} --port=6006

# You can view the tensorboard in the browser url: http://localhost:6006/

# Useful command line to have if tensorboard is misbehaving: kill $(ps -e
```

In [ ]:
```
# TODO: Sample Profiling - Online Inference:

# Vary this from 10, 100, 1000 to simulate multiple online inference
loop_index = [10, 100, 1000]

for idx in loop_index:

    tf.profiler.experimental.start(log_dir_run)

    for i in range(idx):
        # Actual online inference
        # TODO: Your Code Here
        for batch in ds_val.batch(1).take(1):
            image = batch["image"]
            image = tf.cast(image, tf.float32) / 255.0
            pred = model.predict(image)

    tf.profiler.experimental.stop()
```

In [ ]:
```
# TODO: Sample Profiling - Batch Inference:

# We would only perform batch inference for a subset of validation set i.
# using different batch sizes of 20, 40, 100, 200

# Decides the size of the batch. Try: 20, 40, 100, 200
batch_size = [20, 40, 100, 200]

for batch in batch_size:
    # Starts Profile logging
    tf.profiler.experimental.start(log_dir_run)

    for batch in ds_val.batch(batch).take(1000 // batch):
        image = batch["image"]
        image = tf.cast(image, tf.float32) / 255.0
        pred = model.predict(image)

    tf.profiler.experimental.stop()
```

Training

In [ ]:
```
# Setup for model training
from tensorflow.keras import Model, datasets
from tensorflow.keras.models import Sequential
from tensorflow.keras.losses import categorical_crossentropy
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.layers import Dense, Flatten, Conv2D, AveragePoolin
from keras.callbacks import ReduceLROnPlateau
```

```
from tensorflow.keras.metrics import TopKCategoricalAccuracy

train_dir = os.path.abspath(os.path.join('train_data', datetime.datetime.
pathlib.Path(train_dir).mkdir(exist_ok=True, parents=True)



# Using early stopping to monitor validation accuracy
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        # Stop training when `val_loss` is no longer improving
        monitor="val_loss",
        # "no longer improving" being defined as "no better than 1e-2 les
        min_delta=1e-4,
        # "no longer improving" being further defined as "for at least 2
        patience=2,
        verbose=1,
    ),
    tf.keras.callbacks.TensorBoard(log_dir=train_dir, histogram_freq=1)
]
```

In [ ]:

```
def new_model(): # CREATED THIS SO WE CAN GET NEW MODELS TO TRAIN FOR EAC
    # Basic CNN model
    train_model = Sequential()

    # conv1
    train_model.add(Conv2D(32, (5, 5), input_shape=(64, 64, 3), activatio
    train_model.add(Conv2D(32, (5,5),activation='relu'))
    train_model.add(MaxPooling2D(pool_size=(2, 2)))
    train_model.add(Conv2D(64, (3,3), activation='relu'))
    train_model.add(Conv2D(64, (3,3), activation='relu'))
    train_model.add(MaxPooling2D(pool_size=(2, 2)))
    train_model.add(Conv2D(64, (3,3), activation='relu'))
    train_model.add(Conv2D(128, (3,3), activation='relu'))
    train_model.add(MaxPooling2D(pool_size=(2, 2)))
    train_model.add(Flatten())

    # fc1
    train_model.add(Dense(256, activation='relu'))

    # fc2
    train_model.add(Dense(200, activation='softmax'))

    train_model.compile(loss='categorical_crossentropy', optimizer=tf.ker
    # TODO: Consider looking at different optimizers and learning rate se
    #train_model.summary() comment out for now
    return train_model
```

In [ ]:

```
# TODO: Attempt to train your own model with different batch sizes
# TODO: See how long this takes without a GPU on your VDI or 2050 Coover
# TODO: THEN log in to your GPU VM, set ENABLE_GPU = False in the very fi
# TODO: Make sure you have exported the LD_LIBRARY_PATH as the lab manual

def normalize_img(image, label):
    return tf.cast(image, tf.float32) / 255., label

def to_categorical(image, label):
    label = tf.one_hot(tf.cast(label, tf.int32), 200)
    return tf.cast(image, tf.float32), tf.cast(label, tf.int64)
```

```
ds_re = tiny_imagenet_builder.as_dataset(as_supervised=True)
ds_retrain, ds_reval = ds_re["train"], ds_re["validation"]

ds_retrain = ds_retrain.cache().shuffle(1024)
ds_reval = ds_reval.cache().shuffle(1024)

ds_retrain = ds_retrain.map(normalize_img, num_parallel_calls=tf.data.AUT
ds_reval = ds_reval.map(normalize_img, num_parallel_calls=tf.data.AUTOTUN

ds_retrain = ds_retrain.map(to_categorical, num_parallel_calls=tf.data.AU
ds_reval = ds_reval.map(to_categorical, num_parallel_calls=tf.data.AUTOTU

epoch_size = 20

best_top1 = {'batch_size': None, 'accuracy': 0.0}
best_top5 = {'batch_size': None, 'accuracy': 0.0}

for batch_size in [32, 64, 128]:
    # Setup our batched datasets
    print(f"\nTraining with batch size: {batch_size}")
    # TODO: Your Code Here
    train_dataset = ds_retrain.batch(batch_size).prefetch(tf.data.AUTOTUN
    val_dataset = ds_reval.batch(batch_size).prefetch(tf.data.AUTOTUNE)

    train_model = new_model()

    history = train_model.fit(
            train_dataset,
            epochs=epoch_size,
            validation_data=val_dataset,
            callbacks=[callbacks],
            verbose=1
        )
    last_val_top1_accuracy = history.history['val_accuracy'][-1]
    last_val_top5_accuracy = history.history['val_top_5_accuracy'][-1]

    # Get best top-1 overall (kind of inaccurate as the val accuracy drop
    if last_val_top1_accuracy > best_top1['accuracy']:
        best_top1['accuracy'] = last_val_top1_accuracy
        best_top1['batch_size'] = batch_size

    # Update best Top-5 accuracy (kind of inaccurate as the val accuracy
    if last_val_top5_accuracy > best_top5['accuracy']:
        best_top5['accuracy'] = last_val_top5_accuracy
        best_top5['batch_size'] = batch_size

print("\nSummary of Best Accuracies:")
print(f"Best Top-1 Validation Accuracy: {best_top1['accuracy']:.4f} (Batc
print(f"Best Top-5 Validation Accuracy: {best_top5['accuracy']:.4f} (Batc
```

```
Training with batch size: 32
/home/tbibus/CprE587/lab01/cpre-487-587-lab1/lab1_venv/lib64/python3.9/sit
e-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: D
o not pass an `input_shape`/`input_dim` argument to a layer. When using Se
quential models, prefer using an `Input(shape)` object as the first layer
in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Epoch 1/20
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 21s 5ms/step - accuracy: 0.0178 - loss: 5.1
202 - top_5_accuracy: 0.0689 - val_accuracy: 0.0637 - val_loss: 4.6708 - v
al_top_5_accuracy: 0.1968
Epoch 2/20
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.0760 - loss: 4.5
669 - top_5_accuracy: 0.2182 - val_accuracy: 0.1042 - val_loss: 4.3266 - v
al_top_5_accuracy: 0.2841
Epoch 3/20
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 16s 5ms/step - accuracy: 0.1212 - loss: 4.2
298 - top_5_accuracy: 0.3006 - val_accuracy: 0.1398 - val_loss: 4.0970 - v
al_top_5_accuracy: 0.3313
Epoch 4/20
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.1531 - loss: 3.9
947 - top_5_accuracy: 0.3586 - val_accuracy: 0.1566 - val_loss: 3.9622 - v
al_top_5_accuracy: 0.3721
Epoch 5/20
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.1857 - loss: 3.7
927 - top_5_accuracy: 0.4073 - val_accuracy: 0.1763 - val_loss: 3.8363 - v
al_top_5_accuracy: 0.3979
Epoch 6/20
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.2115 - loss: 3.6
206 - top_5_accuracy: 0.4458 - val_accuracy: 0.1959 - val_loss: 3.7297 - v
al_top_5_accuracy: 0.4259
Epoch 7/20
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.2374 - loss: 3.4
692 - top_5_accuracy: 0.4794 - val_accuracy: 0.2110 - val_loss: 3.6504 - v
al_top_5_accuracy: 0.4413
Epoch 8/20
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 16s 5ms/step - accuracy: 0.2571 - loss: 3.3
417 - top_5_accuracy: 0.5060 - val_accuracy: 0.2172 - val_loss: 3.6190 - v
al_top_5_accuracy: 0.4465
Epoch 9/20
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 16s 5ms/step - accuracy: 0.2784 - loss: 3.2
203 - top_5_accuracy: 0.5317 - val_accuracy: 0.2160 - val_loss: 3.6164 - v
al_top_5_accuracy: 0.4491
Epoch 10/20
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.2964 - loss: 3.1
055 - top_5_accuracy: 0.5546 - val_accuracy: 0.2291 - val_loss: 3.5754 - v
al_top_5_accuracy: 0.4639
Epoch 11/20
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 16s 5ms/step - accuracy: 0.3167 - loss: 2.9
910 - top_5_accuracy: 0.5786 - val_accuracy: 0.2304 - val_loss: 3.6061 - v
al_top_5_accuracy: 0.4690
Epoch 12/20
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 16s 5ms/step - accuracy: 0.3365 - loss: 2.8
871 - top_5_accuracy: 0.6014 - val_accuracy: 0.2361 - val_loss: 3.5742 - v
al_top_5_accuracy: 0.4720
Epoch 13/20
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 16s 5ms/step - accuracy: 0.3547 - loss: 2.7
826 - top_5_accuracy: 0.6219 - val_accuracy: 0.2336 - val_loss: 3.6305 - v
al_top_5_accuracy: 0.4723
Epoch 14/20
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 16s 5ms/step - accuracy: 0.3737 - loss: 2.6
845 - top_5_accuracy: 0.6427 - val_accuracy: 0.2341 - val_loss: 3.6356 - v
al_top_5_accuracy: 0.4698
Epoch 14: early stopping

Training with batch size: 64
Epoch 1/20
```

```
I0000 00:00:1757352933.733971  173198 asm_compiler.cc:369] ptxas warning :
Registers are spilled to local memory in function 'triton_gemm_dot_901', 1
24 bytes spill stores, 124 bytes spill loads

I0000 00:00:1757352933.804194  173200 asm_compiler.cc:369] ptxas warning :
Registers are spilled to local memory in function 'triton_gemm_dot_901', 1
12 bytes spill stores, 112 bytes spill loads

I0000 00:00:1757352934.555475  173203 asm_compiler.cc:369] ptxas warning :
Registers are spilled to local memory in function 'triton_gemm_dot_746', 3
12 bytes spill stores, 280 bytes spill loads

I0000 00:00:1757352934.966935  173197 asm_compiler.cc:369] ptxas warning :
Registers are spilled to local memory in function 'triton_gemm_dot_746', 8
8 bytes spill stores, 88 bytes spill loads
```

```
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 19s 8ms/step - accuracy: 0.0227 - loss: 5.1
100 - top_5_accuracy: 0.0780 - val_accuracy: 0.0717 - val_loss: 4.6273 - v
al_top_5_accuracy: 0.2017
Epoch 2/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.0818 - loss: 4.5
378 - top_5_accuracy: 0.2240 - val_accuracy: 0.0990 - val_loss: 4.3920 - v
al_top_5_accuracy: 0.2626
Epoch 3/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.1142 - loss: 4.2
884 - top_5_accuracy: 0.2887 - val_accuracy: 0.1304 - val_loss: 4.1641 - v
al_top_5_accuracy: 0.3194
Epoch 4/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.1442 - loss: 4.0
714 - top_5_accuracy: 0.3413 - val_accuracy: 0.1580 - val_loss: 3.9821 - v
al_top_5_accuracy: 0.3690
Epoch 5/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.1721 - loss: 3.8
832 - top_5_accuracy: 0.3842 - val_accuracy: 0.1763 - val_loss: 3.8628 - v
al_top_5_accuracy: 0.3913
Epoch 6/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.1953 - loss: 3.7
272 - top_5_accuracy: 0.4233 - val_accuracy: 0.1900 - val_loss: 3.7591 - v
al_top_5_accuracy: 0.4161
Epoch 7/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.2156 - loss: 3.5
986 - top_5_accuracy: 0.4502 - val_accuracy: 0.2029 - val_loss: 3.7080 - v
al_top_5_accuracy: 0.4299
Epoch 8/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.2339 - loss: 3.4
925 - top_5_accuracy: 0.4756 - val_accuracy: 0.2079 - val_loss: 3.6583 - v
al_top_5_accuracy: 0.4383
Epoch 9/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.2496 - loss: 3.3
939 - top_5_accuracy: 0.4936 - val_accuracy: 0.2171 - val_loss: 3.6094 - v
al_top_5_accuracy: 0.4494
Epoch 10/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.2663 - loss: 3.2
988 - top_5_accuracy: 0.5143 - val_accuracy: 0.2244 - val_loss: 3.5819 - v
al_top_5_accuracy: 0.4574
Epoch 11/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.2792 - loss: 3.2
150 - top_5_accuracy: 0.5338 - val_accuracy: 0.2287 - val_loss: 3.5486 - v
al_top_5_accuracy: 0.4643
Epoch 12/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.2951 - loss: 3.1
290 - top_5_accuracy: 0.5504 - val_accuracy: 0.2353 - val_loss: 3.5130 - v
al_top_5_accuracy: 0.4746
Epoch 13/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.3095 - loss: 3.0
469 - top_5_accuracy: 0.5691 - val_accuracy: 0.2372 - val_loss: 3.5465 - v
al_top_5_accuracy: 0.4722
Epoch 14/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.3214 - loss: 2.9
756 - top_5_accuracy: 0.5853 - val_accuracy: 0.2401 - val_loss: 3.5261 - v
al_top_5_accuracy: 0.4757
Epoch 14: early stopping

Training with batch size: 128
Epoch 1/20
```

```
I0000 00:00:1757353100.037996  173978 asm_compiler.cc:369] ptxas warning :
Registers are spilled to local memory in function 'triton_gemm_dot_746', 2
52 bytes spill stores, 220 bytes spill loads

I0000 00:00:1757353100.695030  173985 asm_compiler.cc:369] ptxas warning :
Registers are spilled to local memory in function 'triton_gemm_dot_746', 8
8 bytes spill stores, 88 bytes spill loads

I0000 00:00:1757353100.805383  173982 asm_compiler.cc:369] ptxas warning :
Registers are spilled to local memory in function 'triton_gemm_dot_901', 1
16 bytes spill stores, 116 bytes spill loads

I0000 00:00:1757353101.285273  173983 asm_compiler.cc:369] ptxas warning :
Registers are spilled to local memory in function 'triton_gemm_dot_901', 4
bytes spill stores, 4 bytes spill loads
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 17s 14ms/step - accuracy: 0.0175 - loss: 5.15
13 - top_5_accuracy: 0.0687 - val_accuracy: 0.0511 - val_loss: 4.8074 - va
l_top_5_accuracy: 0.1636
Epoch 2/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.0655 - loss: 4.697
8 - top_5_accuracy: 0.1905 - val_accuracy: 0.0874 - val_loss: 4.4953 - val
_top_5_accuracy: 0.2382
Epoch 3/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.0995 - loss: 4.410
0 - top_5_accuracy: 0.2588 - val_accuracy: 0.1105 - val_loss: 4.3107 - val
_top_5_accuracy: 0.2835
Epoch 4/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.1256 - loss: 4.212
8 - top_5_accuracy: 0.3086 - val_accuracy: 0.1275 - val_loss: 4.1852 - val
_top_5_accuracy: 0.3203
Epoch 5/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.1498 - loss: 4.053
1 - top_5_accuracy: 0.3462 - val_accuracy: 0.1520 - val_loss: 4.0277 - val
_top_5_accuracy: 0.3619
Epoch 6/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.1703 - loss: 3.915
6 - top_5_accuracy: 0.3809 - val_accuracy: 0.1652 - val_loss: 3.9476 - val
_top_5_accuracy: 0.3780
Epoch 7/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.1883 - loss: 3.797
1 - top_5_accuracy: 0.4096 - val_accuracy: 0.1766 - val_loss: 3.8894 - val
_top_5_accuracy: 0.3940
Epoch 8/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.2039 - loss: 3.696
8 - top_5_accuracy: 0.4317 - val_accuracy: 0.1810 - val_loss: 3.8230 - val
_top_5_accuracy: 0.4020
Epoch 9/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.2200 - loss: 3.603
1 - top_5_accuracy: 0.4519 - val_accuracy: 0.1879 - val_loss: 3.7902 - val
_top_5_accuracy: 0.4116
Epoch 10/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.2318 - loss: 3.524
5 - top_5_accuracy: 0.4710 - val_accuracy: 0.1990 - val_loss: 3.7548 - val
_top_5_accuracy: 0.4188
Epoch 11/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.2432 - loss: 3.449
3 - top_5_accuracy: 0.4862 - val_accuracy: 0.1967 - val_loss: 3.7538 - val
_top_5_accuracy: 0.4217
Epoch 12/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.2579 - loss: 3.370
3 - top_5_accuracy: 0.5015 - val_accuracy: 0.2097 - val_loss: 3.6710 - val
_top_5_accuracy: 0.4372
Epoch 13/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.2698 - loss: 3.298
1 - top_5_accuracy: 0.5177 - val_accuracy: 0.2150 - val_loss: 3.6578 - val
_top_5_accuracy: 0.4472
Epoch 14/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.2800 - loss: 3.235
8 - top_5_accuracy: 0.5328 - val_accuracy: 0.2215 - val_loss: 3.6285 - val
_top_5_accuracy: 0.4515
Epoch 15/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.2908 - loss: 3.168
1 - top_5_accuracy: 0.5471 - val_accuracy: 0.2277 - val_loss: 3.6086 - val
_top_5_accuracy: 0.4601
Epoch 16/20
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.3010 - loss: 3.107
1 - top_5_accuracy: 0.5590 - val_accuracy: 0.2290 - val_loss: 3.5914 - val
_top_5_accuracy: 0.4660
Epoch 17/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.3112 - loss: 3.055
9 - top_5_accuracy: 0.5700 - val_accuracy: 0.2324 - val_loss: 3.5843 - val
_top_5_accuracy: 0.4655
Epoch 18/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.3250 - loss: 2.985
6 - top_5_accuracy: 0.5831 - val_accuracy: 0.2351 - val_loss: 3.5897 - val
_top_5_accuracy: 0.4693
Epoch 19/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.3353 - loss: 2.926
3 - top_5_accuracy: 0.5952 - val_accuracy: 0.2349 - val_loss: 3.5969 - val
_top_5_accuracy: 0.4681
Epoch 19: early stopping

Summary of Best Accuracies:
Best Top-1 Validation Accuracy: 0.2401 (Batch Size: 64)
Best Top-5 Validation Accuracy: 0.4757 (Batch Size: 64)
```

In [ ]:

```
# TODO: Train your model with 3 different numbers of epochs
batch_size = 32

# Setup your datasets
# TODO: Your Code Here
train_dataset = ds_retrain.batch(batch_size).prefetch(tf.data.AUTOTUNE)
val_dataset = ds_reval.batch(batch_size).prefetch(tf.data.AUTOTUNE)

for epoch_size in [3, 10, 100]:
    # Run training
    # TODO: Your Code Here
    train_model = new_model()

    history = train_model.fit(
            train_dataset,
            epochs=epoch_size,
            validation_data=val_dataset,
            callbacks=[callbacks],
            verbose=1
        )

    # Save the cnn model
    train_model.save(os.path.join(log_dir, f'CNN_TinyImageNet_train_batch
    last_val_top1_accuracy = history.history['val_accuracy'][-1]
    last_val_top5_accuracy = history.history['val_top_5_accuracy'][-1]

    # Update best Top-1 accuracy (kind of inaccurate as the val accuracy
    if last_val_top1_accuracy > best_top1['accuracy']:
        best_top1['accuracy'] = last_val_top1_accuracy
        best_top1['batch_size'] = batch_size

    # Update best Top-5 accuracy (kind of inaccurate as the val accuracy
    if last_val_top5_accuracy > best_top5['accuracy']:
        best_top5['accuracy'] = last_val_top5_accuracy
        best_top5['batch_size'] = batch_size

# Report the best overall accuracies
print("\nSummary of Best Accuracies:")
print(f"Best Top-1 Validation Accuracy: {best_top1['accuracy']:.4f} (Batc
```

```
print(f"Best Top-5 Validation Accuracy: {best_top5['accuracy']:.4f} (Batc
```

```
Epoch 1/3
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 18s 5ms/step - accuracy: 0.0205 - loss: 5.1
059 - top_5_accuracy: 0.0756 - val_accuracy: 0.0752 - val_loss: 4.5527 - v
al_top_5_accuracy: 0.2180
Epoch 2/3
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.0871 - loss: 4.4
696 - top_5_accuracy: 0.2393 - val_accuracy: 0.1202 - val_loss: 4.2277 - v
al_top_5_accuracy: 0.3005
Epoch 3/3
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.1308 - loss: 4.1
346 - top_5_accuracy: 0.3244 - val_accuracy: 0.1488 - val_loss: 3.9844 - v
al_top_5_accuracy: 0.3595
WARNING:absl:You are saving your model as an HDF5 file via `model.save()`
or `keras.saving.save_model(model)`. This file format is considered legacy
. We recommend using instead the native Keras format, e.g. `model.save('my
_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
Epoch 1/10
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 18s 5ms/step - accuracy: 0.0213 - loss: 5.1
100 - top_5_accuracy: 0.0768 - val_accuracy: 0.0757 - val_loss: 4.5948 - v
al_top_5_accuracy: 0.2076
Epoch 2/10
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.0852 - loss: 4.4
849 - top_5_accuracy: 0.2360 - val_accuracy: 0.1160 - val_loss: 4.2327 - v
al_top_5_accuracy: 0.2980
Epoch 3/10
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.1261 - loss: 4.1
576 - top_5_accuracy: 0.3180 - val_accuracy: 0.1409 - val_loss: 4.0571 - v
al_top_5_accuracy: 0.3411
Epoch 4/10
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.1610 - loss: 3.9
241 - top_5_accuracy: 0.3744 - val_accuracy: 0.1669 - val_loss: 3.9041 - v
al_top_5_accuracy: 0.3794
Epoch 5/10
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.1919 - loss: 3.7
339 - top_5_accuracy: 0.4210 - val_accuracy: 0.1853 - val_loss: 3.7713 - v
al_top_5_accuracy: 0.4114
Epoch 6/10
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.2174 - loss: 3.5
690 - top_5_accuracy: 0.4561 - val_accuracy: 0.2035 - val_loss: 3.6549 - v
al_top_5_accuracy: 0.4370
Epoch 7/10
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.2422 - loss: 3.4
248 - top_5_accuracy: 0.4875 - val_accuracy: 0.2150 - val_loss: 3.6124 - v
al_top_5_accuracy: 0.4470
Epoch 8/10
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.2640 - loss: 3.2
916 - top_5_accuracy: 0.5155 - val_accuracy: 0.2291 - val_loss: 3.5269 - v
al_top_5_accuracy: 0.4704
Epoch 9/10
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.2853 - loss: 3.1
729 - top_5_accuracy: 0.5423 - val_accuracy: 0.2271 - val_loss: 3.5588 - v
al_top_5_accuracy: 0.4686
Epoch 10/10
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.3038 - loss: 3.0
631 - top_5_accuracy: 0.5640 - val_accuracy: 0.2290 - val_loss: 3.5397 - v
al_top_5_accuracy: 0.4672
Epoch 10: early stopping
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()`
or `keras.saving.save_model(model)`. This file format is considered legacy
. We recommend using instead the native Keras format, e.g. `model.save('my
_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
Epoch 1/100
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 18s 5ms/step - accuracy: 0.0224 - loss: 5.0
995 - top_5_accuracy: 0.0775 - val_accuracy: 0.0809 - val_loss: 4.5519 - v
al_top_5_accuracy: 0.2179
Epoch 2/100
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.0887 - loss: 4.4
559 - top_5_accuracy: 0.2433 - val_accuracy: 0.1192 - val_loss: 4.2384 - v
al_top_5_accuracy: 0.2979
Epoch 3/100
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.1265 - loss: 4.1
604 - top_5_accuracy: 0.3185 - val_accuracy: 0.1440 - val_loss: 4.0416 - v
al_top_5_accuracy: 0.3448
Epoch 4/100
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.1620 - loss: 3.9
269 - top_5_accuracy: 0.3727 - val_accuracy: 0.1753 - val_loss: 3.8401 - v
al_top_5_accuracy: 0.3911
Epoch 5/100
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.1937 - loss: 3.7
280 - top_5_accuracy: 0.4214 - val_accuracy: 0.1848 - val_loss: 3.7463 - v
al_top_5_accuracy: 0.4120
Epoch 6/100
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.2171 - loss: 3.5
730 - top_5_accuracy: 0.4560 - val_accuracy: 0.2032 - val_loss: 3.6771 - v
al_top_5_accuracy: 0.4267
Epoch 7/100
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.2389 - loss: 3.4
347 - top_5_accuracy: 0.4864 - val_accuracy: 0.2198 - val_loss: 3.5685 - v
al_top_5_accuracy: 0.4518
Epoch 8/100
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.2599 - loss: 3.3
147 - top_5_accuracy: 0.5134 - val_accuracy: 0.2297 - val_loss: 3.5221 - v
al_top_5_accuracy: 0.4633
Epoch 9/100
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.2793 - loss: 3.2
012 - top_5_accuracy: 0.5358 - val_accuracy: 0.2342 - val_loss: 3.4836 - v
al_top_5_accuracy: 0.4744
Epoch 10/100
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.2969 - loss: 3.0
982 - top_5_accuracy: 0.5571 - val_accuracy: 0.2396 - val_loss: 3.4772 - v
al_top_5_accuracy: 0.4772
Epoch 11/100
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.3151 - loss: 2.9
900 - top_5_accuracy: 0.5799 - val_accuracy: 0.2401 - val_loss: 3.4851 - v
al_top_5_accuracy: 0.4829
Epoch 12/100
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 15s 5ms/step - accuracy: 0.3320 - loss: 2.8
991 - top_5_accuracy: 0.5960 - val_accuracy: 0.2460 - val_loss: 3.4899 - v
al_top_5_accuracy: 0.4818
Epoch 12: early stopping
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()`
or `keras.saving.save_model(model)`. This file format is considered legacy
. We recommend using instead the native Keras format, e.g. `model.save('my
_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
```

```
Summary of Best Accuracies:
Best Top-1 Validation Accuracy: 0.2460 (Batch Size: 32)
Best Top-5 Validation Accuracy: 0.4818 (Batch Size: 32)
```

Above and Beyond

In [27]:
```
# Benchmark our dataset to make sure loading our data isn't a bottleneck
# (This can be skipped since it can take a bit and is't all that importan

# tfds.benchmark(ds_train.batch(32), batch_size=32, num_iter=2**20)
```

In [28]:
```
# Explore new models to find a higher-accuracy model. Does the new model
```